ARC-AGI-3 Part 2：工程实现与 debug 记录（大白话版）

Bo Hou（与 Claude 协作整理）

2026-10

0 这篇讲什么

Part 1 讲的是“题目是什么、两层世界模型怎么形式化、别人怎么做”。这篇接着往下，讲**真正动手以后**发生了什么：代码怎么一版一版改的、每次 debug 碰到什么坑、为什么这么改、我们俩中间讨论过哪些取舍。

不讲数学推导（GDN 那部分单独放在番外篇）。这里全是工程：哪里慢、哪里爆、哪里是我们自己的 bug。

阅读提示：

- “**Bo 定的**”= 方向和规矩，是 Bo 拍板的。
- “**Claude 的优化**”= 实现层面 Claude 自己加的改动，每条都写了“为什么”。
- 数字都是实测或模拟测出来的，注明了出处；GPU 正式跑还没开始，所以有些数字只是模拟器上的。

1 起点：要搭一个什么东西

1.1 一句话目标

让一个**离线**的 ~30B 开源模型（Qwen）自己玩没见过的游戏：看画面 → 猜规则 → 把规则写成 Python 代码（世界模型）→ 用代码在 CPU 上搜索出一串动作 → 执行。玩完一个游戏，把有用的代码沉淀进一个共享函数库，下个游戏接着用。

对应 Part 1 的两层：

- **内层**（每个游戏一份）：Qwen 写的 `Model` 类（预测下一帧）+ `plan`（搜索动作序列）+ 目标猜想 `is_win`。
- **外层**（跨游戏共享）：模型本身 + 共享库 `lib` + 提示词/流程。

1.2 Bo 定的规矩（一直没变过）

1. **公开的 25 个游戏源码不能拿来学**：“那个代码用作测试和 future 可能 rl，不对 rl 都不行，那个是标准答案，都不是 label，不然学个寂寞”。也不能用来训练：“也不用，用了也是作弊”。
2. 只有 Bo 和 Claude 能拿生成的代码去对照源码，而且只在 12 个开发游戏上，目的是改进通用的提示词。Qwen 永远看不到源码、游戏名、人类基线。
3. 另外 13 个游戏是留出集，**谁都不看**。
4. 要合成游戏，只能用开源模型写（不用 ChatGPT，条款问题）。

5. 只用离线 ~30B 模型，不走“大模型当老师”路线（工程量太大）。
6. 设计原则：“**能用程序跑的就别自己理解自然语言推理，能形式化程序跑的就形式化**”；“**外层程序不 work 的，就去掉**”。
7. 一个游戏的所有关卡共用一**个对话**；思考长度要有限（离线比赛时间有限）。
8. GPU 机器（RunPod）不经 Bo 同意不许停、不许删、不许花钱。

1.3 硬件

场景	硬件	说明
Kaggle 比赛	1 张 RTX PRO 6000（96 GB），断网，总共 9 小时	110 个隐藏游戏一起跑
我们自己实验	RunPod A100 80 GB	持久盘 /workspace，vLLM 起服务

2 第一步：从人类玩的日志里挖规则

2.1 做了什么

在写 harness 之前，先拿**人类玩家的操作日志**（每一步按了什么、画面怎么变，不是游戏源码）做统计，看人在陌生游戏里有哪些共同习惯。目的是把靠得住的习惯写成几句“建议”，放进 Qwen 的提示词里当先验。

一共看了六条（数字都是跨游戏的中位数）：

编号	问的问题	统计结果（大白话）
R1	人多快把每个动作都试一遍?	动作 1–6 一般在开局前 3–9 步就第一次用到，几乎没有整局不用的；动作 7（撤销）很晚才用，约 7 成的局根本不用
R2	点击是不是点在物体上？点了会不会变?	一开始算出来接近 0（统计 bug，见下）；修好后 0.86–1.0：人基本都点物体，点了附近就变
R3	一个动作没效果，人还会再按它吗?	会，但几乎白按：复查里这种“没效果后再按同一个”共 4210 次，画面真变了的只有约 1%
R4	一个动作有效果，人会接着按吗?	大约一半多会接着按（≈ 0.5–0.65）——有效的动作通常要连着用
R5	游戏结束（game over）后人会马上重开吗?	约 1/3
R6	人在什么时候停下来想?	以普通一步的间隔为基准：第一次用某个动作前约 1.6 倍，**过关后约 6 倍**

原来还有一条 R7，Bo 看了觉得没用，去掉了。

2.2 能量条检测器

很多游戏的画面边上有一根细条：每按一次动作就少一格，用完就 game over。它是**步数/能量计数器**（游戏界面的一部分，HUD），不是游戏里的物体。它带来两个麻烦：

- 统计“这一步有没有效果”时，它每步都在变，于是几乎每一步都算“有效果”，R3 这类统计就脏了；
- 以后 Qwen 写世界模型时，如果不知道它是计数器，会把它当成一个要去理解、要去控制的物体，白费力气。

所以要一个程序，**只看画面**就能把它找出来。前后试了三种：

方法	怎么找	结果
频率法	在人类日志里，哪个像素在 80% 以上的步里都变，就算 HUD	**一个像素都没找到**（所有游戏都是 0）。原因：能量条每步只变一两个像素，而且每次变的是不同位置，单个像素的变化频率很低
D1 形状法	只看第一帧：靠近边缘（6 像素内）、很细（≤3）、很长（≥8）的同色条	中位数精确率、召回率都是 1.0；22 个有 HUD 的游戏全找到。错的地方：3 个本来没有 HUD 的游戏也报了；有 2 个游戏的 HUD 不是细条形，几乎没找到
D2 形状 + 变化确认	D1 的候选，再要求开局前几步里它确实小幅变化	**反而更差**：22 个只找到 12 个，像素召回中位数掉到 0.02。原因：开局几步里能量条常常还没动

怎么知道对不对：游戏引擎会把“界面层”（HUD）单独画，我们拿它当标准答案，**只用来给检测器打分**；检测器本身只看画面，Qwen 也看不到这个答案。

最后用到 harness 里的是形状 + 在线确认的思路（见 v6 的感知模块）：在边缘 3 像素内的细条，如果大多数步都只变几个像素（≤4），就当计数器，每步只报一句 `counter`，不再算成物体变化。和 D2 的区别是：不在开局几步就下结论，而是边玩边累计证据。

2.3 最后进了提示词的只有四句

1. 开局先把每个可用动作都试一次（撤销可以晚点）。——来自 R1
2. 点击先点物体，一个物体点一次，空白处放后面。——来自 R2
3. 一个动作没效果（计数器变不算），同样情况下别再按。——来自 R3
4. 边上每步变一点的细条通常是步数/能量计数器，是有限资源，不是游戏物体。——来自能量条检测（2.2）

R4–R6 只是描述人的行为，对“怎么少走步”没有直接帮助，就没放。R6（过关后人会停下来想很久）后来间接体现在“让模型自己决定这一步要不要深度思考”上。

2.4 教训

1. **数字反常先查自己的代码。** R2 接近 0 明显不合理（人点东西几乎总会变）。一查是 Claude 的统计 bug：点击坐标是 `(x, y)` = (列, 行)，统计时当成了 (行, 列)，等于在看别的位置。Bo：“果然还是得我 review 才行”。之后的约定是：**下结论之前先核实**。
2. **“没变化”要先定义清楚。** 计数器每步都在变，不剔掉的话，几乎每一步都算“有效果”，R3 这类统计就是错的。所以先要有能量条检测器（2.2），后来直接变成了 v6 感知模块里的自动计数器检测。
3. **检测器要有标准答案打分，别凭感觉。** 频率法看起来很合理，打分才发现一个都找不到；D2 多加一道确认，直觉上应该更准，打分发现反而更差。
4. **复查要看原始次数，不只看比例。** R3 第一次只有一个比例，看不出“再按一次”到底有没有用；复查数出 4210 次里只有 1% 有变化，结论才站得住。
5. **跨游戏中位数会掩盖差异。** 有的游戏只能点击，有的只有方向键；每个游戏的表差别很大，中位数只能当粗略建议，所以提示词里写的是“advice, not law”（建议，不是定律）。
6. **人的规则很浅。** Bo 的判断：“从人类那里拿到的 rule 感觉还是很浅”。这些规则只能帮开局少走几步弯路，真正决定成败的是“搜索成功率”——模型写出对的代码，CPU 能搜出通关路线。所以后面精力全放在 harness 上。

3 Round 1：用 27B 稠密模型跑第一轮

3.1 当时的流程

每一步都把画面、改动、物体列表等拼成一个大提示词丢给 Qwen3.8-27B，让它回复动作，或者回复 `Model` 代码；代码写出来以后：

- **fit**：程序把模型在所有已发生的步骤上重放一遍，算预测准确率（Claude 的设计：用程序算“模型对不对”，不让 LLM 自己判断）。
- 一次采样 k 份模型（Round 1 是 4 份），**按 fit 分数挑最好的**。怎么做到每份不一样：同一个提示词同时发 k 个请求，模型生成每个 token 时是按概率抽签的（temperature 0.6、top_p 0.95、top_k 20），所以每份代码都不同；不设随机种子，也不改提示词。到 v6.1，默认 3 份，并按当时有几个游戏同时在玩自动调整，最多 6 份。
- **plan**：在子进程里跑 Qwen 写的搜索代码，得到动作序列，一步一步执行，每步都对照预测，错了就停下来。
- 一个游戏结束后，让 Qwen 整理（refactor）共享库。

3.2 一晚上跑下来的结果：12 个游戏里 5 个 0 分，到处超时

Bo 让 Claude 判断“是代码问题还是设计思路问题”。拆开看，大部分是**我们这边的问题**：

现象	原因	怎么改的
模型每一步都想很久	思考档位参数放错了地方（见表下说明），27B 的服务**直接忽略**，于是永远按最高档思考	换个位置放
大量超时、重试	客户端 600 s 超时 + 自动重试，一次长思考被重复跑好几遍	超时放到 1800 s，超时不重试

现象	原因	怎么改的
ft09 模型把目标猜错了	**Claude 的 bug**：harness 只给了动作前后两帧，把中间的**动画帧丢了**。ft09 的提示信息只出现在动画里，结果在 Qwen 看来“什么都没变”	保留动画帧
共享库从 75 行膨胀到 1085 行	每个游戏结束都往库里加东西，不管有没有用、对不对	加“门槛”（见下）
一个关卡只有 model.py，没有 plan	思考太长、上下文太大，到 plan 之前时间就用完了	见下一章

“参数放错地方”是什么意思。 程序每次请模型回答，都是发一个 JSON 请求给 vLLM 服务。同一个参数可以放在两个位置：

```
{ "model": "...", "messages": [...],
  "reasoning_effort": "high",                          # 顶层
  "chat_template_kwargs": {"reasoning_effort": "high"} # 里层
}
```

- **顶层**：和 model、messages 并列。这是 OpenAI 接口的标准写法。但这个 vLLM 服务收到后**不报错、也不用**，相当于没写。
- **里层**：塞进 chat_template_kwargs，这里的参数会交给 Qwen 的“聊天模板”（把对话拼成模型真正看到的那段文字的程序），模板会把思考档位写进去，模型才会照做。

我们一开始写在顶层，所以不管设什么，模型都按默认的最高档思考，每一步想很久。改到里层就好了（顺带发现 27B 不接受 “high” 这个档位，会报 400 错误）。

Claude 的优化：共享库的门槛。 为什么：库是外层，库里垃圾越多，每个游戏的提示词越长、模型越容易被带偏。规则：

- 只有 fit ⩾ 0.8（模型被验证过）的代码才能进库；
- 一次最多加 200 行；改完的库必须能 import 成功；
- 连续 3 个游戏没被用到的函数删掉（几个核心函数除外）——这正是 Bo 的“外层程序不 work 的，就去掉”；
- 提示词里只放函数签名，不放函数体。为什么：省 token，Qwen 需要时自己去调用就行。

3.3 两个讨论

“杀鸡用牛刀”。Bo 指出：像 BFS 这种程序能跑的事，为啥要让 LLM 用自然语言推理？算力要仔细分配，能分给 CPU 的就分出去。于是定下：LLM 只负责“想规则、写代码”，搜索、校验、统计全交给 CPU。

“让 AI 自己决定要多少算力”。Claude 加了 `next: fast/think`：Qwen 每次回复时自己说下一步要不要深度思考。为什么：简单的步骤（比如照着计划走）不需要思考，省下的时间给难的步骤。

4 换成 MoE 模型，以及“想太长被截断”

4.1 为什么换

模型	单路速度	4 路并发
Qwen3.8-27B（稠密）	≈ 28 token/s	—
Qwen3.6-35B-A3B-FP8（MoE，每次只激活 ~3B）	≈ 155 token/s	≈ 232 token/s

快 5 倍多。Bo 的判断：“如果 code 差不多，就 moe 吧，感觉不会差太大”。

4.2 MoE 冒烟测试：23 次调用里 20 次被截断

MoE 只有“开/关思考”一个开关，没有思考档位。它一开思考就停不下来，8192 token 上限一到，**回答还没开始写就被截断了**，等于白想。

Claude 的优化：思考预算 + “抢救”。 思考上限设成 4096；如果被截断，把它已经想的最后一段拿出来，再发一次**不思考**的请求：“你刚才想到这里了，现在直接给答案”。为什么：已经花掉的思考不浪费，而且不用再想一遍。模拟测试 110 次截断全部抢救成功。

4.3 Bo 关于“别人怎么解决想太长”的问题

查了一圈：有钱用前沿 API 的方案不在乎（思考随便长）；离线方案（比如 Franzen）是**多轮对话 + 前缀缓存**，每一轮只想一点点，靠对话历史累积。我们后面就走这条路。

5 v4：一个游戏一个对话 + CPU 异步计划

5.1 Bo 指出的根本问题

当时每一步都**重新开一个对话**、重新拼完整提示词。Bo：“我们现在每一步都重新开一个？是一个游戏的每一步都这样子？这明显不对吧……肯定是一个游戏的不同关卡是共享一个对话”。

还有一条：“llm 回应的时候比如跑程序，这个时候可以释放 gpu，然后过 30 秒之后来看运行结果，而不是一直 hung 着，等 cpu 结果”。

5.2 v4 的做法

- 一个游戏 = 一个对话，所有关卡接着聊。每条新消息只带**新的东西**（发生了什么、新步骤、当前画面）。
- Qwen 回复一个 `plan` 代码块后，程序在后台 CPU 上跑它，**GPU 去服务别的游戏**；30 秒后问 Qwen：“还在跑，最新输出是这些，等还是换？”Qwen 回 `wait` 就继续等，回别的就杀掉计划。
- 对话太长时，从最早的地方删掉几轮。

5.3 v4 冒烟测试（ar25，20 分钟）：59 次调用里 33 次报错

报错都是 vLLM 返回 400：“上下文太长”（≥ 127k token）。原因是**Claude 估 token 数的方法错了**：用“字符数 ÷ 3”估，但我们的画面是一串十六进制数字，Qwen 的分词器**每个数字就是一个 token**，实际是估计值的约 3 倍。程序以为还很短，其实早就爆了。

好消息是在报错的缝隙里它还是动起来了：106 个动作、写了 5 版模型、最后一版在已发生的步骤上预测 100% 正确、执行了一个 55 步的计划。但一直没写 `goal:`（目标猜想）这一行。

Claude 的修复：先用保守估计（约 1.6 字符/token）顶上，再加一个兜底——服务器真报“太长”，就从最早处删几轮重试。

6 v5：画面不再堆在对话里

v4 能跑，但对话里堆了几十张 64×64 的画面，每张上千 token，很快就满。

Claude 的优化（v5）：

1. **对话里只留当前画面**。 下一条消息来的时候，上一张画面被替换成一行占位符。所有画面程序都存着，Qwen 想看旧画面就写一个 `look` 代码块（在 CPU 上跑的 Python，比如 `print(grid(frames[3], 0, 15, 20, 40))`）。为什么：画面是最占地方的东西，而 Qwen 绝大多数时候只需要看当前这张。
2. **用服务器的真实数字校准 token 估计**。 每次请求，服务器都会返回真实的 `prompt_tokens`；程序拿它反推比例（数字按 1 个 token 算，其他字符按校准出来的比例），再留 4–8% 余量。为什么：v4 的教训就是“估错 3 倍”，干脆不估，用服务器给的真值。
3. **超长时一次性删一大块**，并且删的是真实对话，不是临时副本（v4 只删了副本，下次还会爆）。
4. **沙箱**。 Qwen 写的代码在单独的低权限用户下跑，有内存上限、CPU 时间上限、自己的进程组（杀进程时子进程一起杀）；再加一层静态检查：代码里出现 `import os`、`subprocess`、`open(` 之类直接不跑，并告诉 Qwen 原因。为什么：Qwen 写的代码是不可信的，误删文件或者把机器跑死都可能发生。
5. **一个游戏崩了不影响其他游戏**：异常写进 error.txt，批次继续。

还做了几轮“审稿人”代理复查代码（都只允许看开发游戏）。

一次违规要说清楚：有一次审稿代理在留出游戏 bp35 上跑了随机动作测试。虽然没看源码、没改任何东西，但违反了“留出集谁都不看”。之后检查脚本加了硬限制：非开发游戏直接拒绝运行；审稿提示里也写死了只准碰哪几个游戏。

7 v6：让每一步描述更“人话”，让调度更满

7.1 用物体而不是像素描述变化（percept.py）

之前告诉 Qwen“哪些像素变了”。问题是 Qwen 要自己把一堆像素坐标拼成“哪个东西动了”，这正是 Bo 说的“能程序跑的别让 LLM 推理”。

Claude 的优化：CPU 感知模块。 程序把画面切成同色连通块（物体），对比前后两帧，直接说：

- `c9 3x3 @(10,12) moved (+0,+3)`（蓝色 3×3 的块向右移了 3 格）
- recolored / grew / shrank / new / gone（变色/变大/变小/出现/消失）
- 边上那种每步少一点的细条（步数计数器）自动识别，只报一个 `counter`，不再每步刷屏；
- 回到以前见过的画面会说“same as frames[k]”。

另外程序统计一张**动作效果表**：每个动作到目前为止都干了什么（比如“动作 1：大多是某个物体向上移 1 格”）。为什么：这些都是确定的计算，CPU 做又快又准；实测每步 2.6–3.6 毫秒。

同一画面的写法	大约多少 token
完整十六进制画面	870–2060
物体列表	150–400

7.2 其他省 token 的地方

- **重复行折叠**：和上一行一样的行写成 `a..b same as row a-1`。背景大片一样的画面能省一大截。
- **画面没变就不重发。**
- **模型检查报告只报变化**：上次已经说过的错误不再重复，只说新错了哪几步，而且用物体语言解释错在哪。

7.3 调度

Claude 的优化：滚动调度。 之前是一批 4 个游戏一起开、一起结束，最慢的那个拖着整批。现在是**永远保持 4 个游戏在玩**，一个结束马上补一个；游戏结束后的“整理共享库”放到后台线程做，不占玩游戏的位置。为什么：GPU 不空闲。

慢 fit 节流：如果模型 fit 一次就要很久（Qwen 写了很慢的代码），就少采样几份。

计时：每个游戏记录时间花在哪：LLM / fit / look / plan / 游戏环境。为什么：真正上 GPU 以后，先看瓶颈在哪再优化，而不是猜。

8 v6.1：前缀缓存的坑（这是这段时间最大的一个 debug）

8.1 前缀缓存是什么（大白话）

多轮对话里，每次发给服务器的提示词 = 之前所有内容 + 一条新消息。前面那一大段上次已经算过了，服务器把算过的结果（KV）存着，这次直接拿来用，只算新加的那一点。这就是**前缀缓存**。条件是：**前面那段必须一字不差。**

普通模型里，KV 按 16 个 token 一块存，每块用“前一块的指纹 + 这 16 个 token”算一个指纹（hash，在 CPU 上算），新请求逐块对指纹，对上的块直接复用。哪怕中间某块改了，**改动之前**的块还能复用。

8.2 Qwen3.6 不一样

Qwen3.6 是**混合模型**：约 3/4 的层是线性注意力（GDN，番外篇推导的那个），只有 1/4 是普通注意力。线性注意力层不存每个 token 的 KV，只存一个**固定大小的状态**（相当于把之前所有内容压成一个矩阵）。

问题就来了：这个状态只能存某个位置的“快照”，vLLM 每个请求只保留**一份**快照，位置在对齐到几百到几千 token 的地方。要复用，就要求**从开头到快照位置全都没变**（vLLM issue #45238）。

而 v5/v6 的做法是“每轮把上一张画面换成占位符”——**每一轮都改了上一条消息**，快照位置之前的内容变了，于是**每一轮缓存都失效**，整段对话从头再算。对话越长越亏。

Bo 中间的问题很到位：“如果是六张图才扔，按理来说不是每次 prompt S 都变啊”——对，问题就出在 v6 是**每一轮都扔**（eager），所以每轮都变。

8.3 Claude 的优化：lazy 模式 + 开机探测

- **lazy（懒删除）**：对话只往后加，不改前面。旧画面先留着，攒到超过 6 张（或者对话太长）时，**一次性**把所有旧画面、旧表格、旧图片清掉。为什么：一次清理只破坏一次缓存，之后又能连续复用好多轮；而 eager 是每轮都破坏。
- **eager（勤删除）**：每轮删，对话最短。适合缓存能容忍中间修改的普通模型，或者根本没缓存。
- **开机自动探测**：程序启动时给服务器发 4 个测试请求，测“只在后面追加”和“改了中间”两种情况下缓存命中多少，自动选 lazy 或 eager，结果写进 PROBE.txt。为什么：vLLM 版本不同行为不同，不靠猜。

模拟器（照着混合模型的缓存规则写的）上的结果：

模式	提示词里来自缓存的比例
eager（v6）	≈ 50–55%
lazy（v6.1）	≈ 85%

也就是 GPU 实际要算的输入 token 大约少了 2/3。

8.4 同一版里的其他改动（都是 Claude 的）

- **等待间隔越来越长**：plan 在 CPU 上跑时，第一次 30 秒后问，之后间隔翻倍（最多 4 倍），但不超过截止时间。为什么：每问一次都要 GPU 推理一次，一个要跑 5 分钟的搜索被问 10 次是浪费。
- **候选数自适应**：同时在玩的游戏少的时候，每次多采样几份模型（最多 6 份）；游戏多的时候少采样。为什么：GPU 空着也是空着。
- **图片上限**：vLLM 默认一个请求最多 16 张图，超了直接报错（JustAdev 就被这个搞挂过游戏）。程序默认最多 8 张；服务器报“最多 N 张”就学到这个上限、删旧图；报别的图片错误就先删最新一张，再不行全删，连错 3 次就关掉图片。
- **不开 MTP（多 token 预测加速）**：vLLM 里前缀缓存 + MTP 会把输出弄坏（vLLM issue #53912）。两个只能选一个，选缓存。
- 日志里记录每次的缓存命中数、缓存在哪里断的，方便上 GPU 后核对。

又过了一遍审稿：查出 9 处小问题（比如图片超限时一轮只删一张、等待间隔可能超过截止时间、服务器没返回缓存细节时显示成 0% 等），都修了。28 个模拟场景全部通过。

9 图片：给不给 Qwen 看图?

这一段讨论最长，按顺序说。

9.1 Bo 的出发点

64×64 的画面，人眼都能看清，程序也能直接给出每个物体的位置颜色。那 Qwen 真的需要图吗？能几个变量说清楚的，就不用图。最小推理成本、最少上下文占用，才是目标。

9.2 成本对比（同一帧）

给 Qwen 的形式	大约 token	谁出力
物体列表（程序算好）	150–400	CPU 2.6–3.6 ms
十六进制文字画面	870–2060	几乎为零
图片（放大到 640×640）	≈ 400	GPU 视觉编码器

为什么要放大：Qwen 的视觉编码器按 14–16 像素一个小块切图，64×64 原图里一个格子只有 1 个像素，会被切进同一个小块里糊掉；放大后一个格子变成 10×10 像素，才分得清。而 token 数由**送进去的像素尺寸**决定（新的编码器大约 32×32 像素 = 1 个 token，所以 640×640 ≈ 400），模型内部没法自己放大。

9.3 Bo 提的“理想做法”

原始 64×64 图在上下文里只占大约 10 个 token；放大到 640×640、再做图像处理，这些都发生在模型**内部**的计算里，跟上下文无关。

Claude 去搜了：压缩成很少 token 的模型是有的（比如 1 个或 2 个 token 表示一张图），但都是“大图压小”，为的是省 token，丢细节；我们要的是“**小图、每个像素都精确**”。现成的、能接在 Qwen3.6 上的，没找到。这个可以当以后 RL / 论文的方向。

9.4 别人怎么做的（看了开源代码）

方案	怎么给图
Franzen（离线第一梯队）	每轮都给图，最近邻放大 **10 倍**；他自己测过，**4 倍→10 倍是图片带来的最大提升**；还给“变化图”和失败时的画面；图留在历史里（只追加，正好配合前缀缓存）
JustAdev	放大 4 倍，最多 12 张（之前超过 vLLM 的 16 张上限把游戏搞挂了）
Retrodict	平时全用文字；只在开局用一个单独的视觉模型看一次图（放大 12 倍）

9.5 现在的做法（本地版，还没上传）

Claude 的实现：每帧一张图 + 变化图。

- 画面变了，就附一张当前画面的图（10 倍放大的思路，上限 1280×640）；
- 同一关里有变化时，再附一张“变化图”：变了的格子画原色，没变的格子涂成深蓝（调色板里没有的颜色，不会混淆）；
- 配合 lazy 模式：图也只往后加，攒多了一次性清。

为什么：Franzen 的实测证据比我们的直觉更可靠；文字画面和物体列表照样都给，图是额外的一路。到底有没有用，等 GPU 跑起来做对照实验（不给图 / 按需看图 / 每帧给图 / 不同放大倍数）。

10 合作方式上的几点

- **Bo 觉得“吞吐量低”**：Bo 故意把中间过程来回贴，是为了理解 Claude 到底做了什么——Claude 做得快，但对 Bo 来说像黑箱，容易跟丢。这是正常的代价；所以这篇文档就是把黑箱打开。
- **Claude 的错误**：R2 统计 bug、丢动画帧、token 估错 3 倍、审稿代理碰了留出游戏——都是 Claude 这边的，Bo review 时抓出来或者事后查出来的。
- **规矩一直是 Bo 定的**：数据边界（哪些游戏能看）、一游戏一对话、CPU 能做的不给 LLM、不 work 的外层就删、不能随便动 GPU 机器。

11 现在在哪、接下来

现状：代码 v6.1 已传到 Google Drive（本地还有一版加了每帧图片，等 Bo 决定要不要传）。所有模拟测试通过。RunPod 还在等 A100。

只有 GPU 才能回答的问题：vLLM 版本和真实的缓存行为（看 PROBE.txt）、图片实际多少 token、沙箱在网络盘上能不能设置权限、Qwen 守不守回复格式、真实吞吐。

接下来：

1. 第一次正式跑：4 个游戏并行，15 分钟后看缓存命中率、时间花在哪；
2. 图片对照实验；
3. 根据结果改提示词，再往 RL 走（GRPO，组内均值做基线）；
4. 写 Part 3 / 论文。

附：版本一览

版本	主要变化	解决的问题
Round 1	每步一个大提示词；fit 挑模型；plan 在子进程跑	跑通流程
v3	换 MoE；思考 4096 + 抢救	太慢、思考被截断
v4	一游戏一对话；CPU 异步 plan + 等待	每步重开对话、GPU 干等
v5	只留当前画面；真实 token 校准；沙箱	上下文爆（估错 3 倍）

版本	主要变化	解决的问题
v6	物体级描述；效果表；滚动调度；计时	LLM 做了程序该做的事；GPU 空闲
v6.1	lazy 删除 + 开机探测；等待退避；图片上限	混合模型缓存每轮失效
本地	每帧图 + 变化图	图片证据（Franzen 10×）

附：链接

- vLLM issue #45238（混合模型前缀缓存只在快照前不变时复用）：https://github.com/vllm-project/vllm/issues/45238
- vLLM issue #53912（混合模型上前缀缓存 + MTP 输出出错）：https://github.com/vllm-project/vllm/issues/53912
- Franzen 方案（图片每轮给、放大 10 倍）：https://github.com/tonghuikang/daniel-franzen-arc-agi-3
- JustAdev（放大 4 倍、最多 12 张图）：https://github.com/JustAdev742/Arc-Agi-3-Kaggle-comp
- Retrodict（文字为主、开局用视觉模型看一次）：https://github.com/ryanbbrown/Retrodict